ClickAndGo

Revision oral del codigo

Guion de la defensa

Asignatura	Programacion Orientada a Objeto Seguro - TI3V21
Evaluacion	Sumativa N3 - Unidad 4: desarrollo de soluciones en Python
Instancia	Revision oral del codigo: 10 minutos de presentacion y 5 de preguntas
Fecha y hora	Martes 13 de octubre de 2026, 21:30 hrs
Lugar	Laboratorio A403 (grupo 05, E-commerce)
Integrantes	Miguel Troncoso y Alexandy Remicinthe
Seccion	114-2A-F2 - INACAP Puente Alto
Docente	Michael Arjel
Repositorio	https://github.com/MiguelTroncoso/ecommerce

Este documento es el guion de lo que cada integrante dice y muestra en pantalla. No se lee completo en voz alta: esta escrito para ensayarlo, para saber que archivo abrir en cada momento y para responder las preguntas del docente.

1. Que se evalua y como llegar preparados

La nota tiene dos mitades. Los 21 puntos de producto se asignan una vez por equipo y corresponden al recorrido del codigo: ubicar los requisitos, seguir una operacion completa, mostrar los cambios de la retroalimentacion y explicar la seguridad. Los 21 puntos de defensa son individuales: el docente elige una parte del codigo al azar y cada integrante la explica por separado, predice el efecto de un cambio, justifica una decision de seguridad y da un ejemplo concreto de uso de inteligencia artificial.

1.1 Checklist antes de entrar al laboratorio

El editor con el repositorio abierto debe estar listo antes del inicio del bloque.

- Llegar 15 minutos antes y abrir el repositorio en el editor.
- Actualizar el repositorio: no debe quedar ningun cambio sin guardar.
- Cerrar notificaciones, correo y cualquier pestana que no se use.
- Dejar el terminal cerrado: en esta evaluacion el programa no se ejecuta.
- Fijar el zoom de fuente en un tamano legible desde el fondo de la sala.
- Abrir las pestanas en el orden de la presentacion.
- Tener a mano este guion, en papel o en el celular.

1.2 Pestanas que deben quedar abiertas, en este orden

#	Archivo	Para que se muestra
1	main.py	Menu, orquestacion y captura de las excepciones de negocio
2	model/producto.py	Clase abstracta: contrato de los cuatro subtipos
3	model/pedido.py	Polimorfismo, confirmacion, pago, despacho y las dos reglas
4	model/cliente.py	Validacion obligatoria del RUT o correo en el setter
5	model/excepciones.py	Las excepciones propias del negocio
6	dao/conexion.py	Esquema de tablas y consultas parametrizadas
7	dao/pedido_dao.py	Transaccion con lineas de detalle
8	servicios/validador.py	Requisito no funcional: validacion de entradas
9	servicios/indicador_dolar.py	Requisito no funcional: API con timeout y respaldo
10	servicios/codigos.py	Mejora: codigos automaticos de clientes y productos
11	README.md	Decisiones de seguridad y uso de IA

2. Reparto de la presentacion

Los dos integrantes presentan y los dos responden preguntas. El reparto busca que cada uno tenga una parte del modelo, una de persistencia y una de seguridad.

Bloque	Tiempo	Quien	Contenido	Archivos que abre
1	0:00 - 1:00	Miguel	Apertura, arquitectura de cuatro capas y agenda	main.py
2	1:00 - 3:30	Alexandy	Los cinco requisitos funcionales de la ficha	producto.py, producto_fisico.py, cliente.py, pedido.py, producto_electronica.py
3	3:30 - 5:30	Miguel	Recorrido completo: registrar el pago de un pedido	main.py, pedido.py, producto.py, pedido_dao.py, producto_dao.py
4	5:30 - 7:30	Alexandy	Requisitos no funcionales y decisiones de seguridad	validador.py, indicador_dolar.py, conexion.py, trabajador.py
5	7:30 - 9:30	Miguel	Cambios de la retroalimentacion y uso de IA	codigos.py, consola.py, README.md
6	9:30 - 10:00	Ambos	Cierre y ofrecimiento de preguntas	-

Si el docente pide cambiar el orden, se pueden invertir los bloques 2 y 4 sin problema: los dos integrantes conocen los cuatro bloques.

3. Guion minuto a minuto

Bloque 1 (0:00 - 1:00) - Miguel: apertura y arquitectura

Buenas noches, profesor. Somos Miguel Troncoso y Alexandy Remicinthe, del grupo 5, caso E-commerce: ClickAndGo. El repositorio que esta en pantalla es el mismo de la evaluacion anterior, corregido con la retroalimentacion que recibimos.

Para ubicarnos rapido, el sistema tiene cuatro capas. ***main.py*** *es el menu de consola: pide los datos y captura los errores de negocio. La carpeta* ***servicios/*** *valida las entradas, genera los codigos automaticos y consulta la API del dolar. La carpeta* ***model/*** *tiene las clases del negocio, con las reglas y las excepciones propias. Y la carpeta* ***dao/*** *guarda todo en SQLite con consultas parametrizadas. Ninguna clase del modelo contiene SQL: si cambia la base de datos, solo se modifica la capa dao.*

En estos diez minutos vamos a hacer cuatro cosas: ubicar los requisitos en el codigo, seguir una operacion completa hasta la consulta SQL, mostrar que cambiamos despues de la retroalimentacion y explicar como el codigo protege los datos. Alexandy parte ubicando los cinco requisitos de la ficha.

En pantalla: main.py, el menu y el bloque que captura las excepciones.

Bloque 2 (1:00 - 3:30) - Alexandy: los cinco requisitos funcionales

Gracias, Miguel. La ficha de ClickAndGo pide cinco requisitos funcionales y estan en cinco lugares concretos del codigo.

Primero, tres subtipos con un metodo que se comporta distinto en cada uno. *En* ***model/producto.py*** *esta la clase abstracta Producto, que declara procesar_entrega() sin implementarlo. Los tres subtipos heredan con super().__init__() y lo sobrescriben: ProductoFisico genera un despacho por transporte y calcula el flete con peso, volumen y distancia; ProductoDigital entrega un enlace y una licencia con flete cero; y ProductoServicio agenda una visita tecnica para una fecha futura.*

Segundo, un dato con validacion obligatoria. *Esta en* ***model/cliente.py****, en el setter del atributo identificador. Si el tipo es RUT, se valida con el algoritmo Modulo 11 en validar_rut(); si es correo, se valida con expresion regular. La validacion esta en el modelo y no en el menu: aunque otro programa use estas clases, no se puede construir un cliente con un RUT invalido.*

Tercero, una transaccion con lineas de detalle. *Un pedido es la clase Pedido y cada linea es un DetallePedido, que congela la cantidad y el precio unitario al momento de la venta. La relacion es de composicion: si se elimina el pedido, sus lineas se eliminan con el, y en la base de datos eso esta declarado con ON DELETE CASCADE. En* ***dao/pedido_dao.py*** *el pedido y sus lineas se insertan dentro de una sola transaccion: o se guardan todos o no se guarda ninguno.*

Cuarto, dos reglas que impiden una operacion, cada una con su excepcion propia. *En* ***model/pedido.py****, confirmar_pedido() recorre las lineas y si no hay stock suficiente lanza StockInsuficienteError. Y despachar() verifica el permiso del trabajador y luego que el pedido este pagado; si no lo esta, lanza PedidoNoPagadoError. Las dos excepciones estan en* ***model/excepciones.py*** *y se capturan en main.py con except ClickAndGoError, que informa el problema y devuelve el control al menu.*

Quinto, un precio que depende de un indicador externo. *La clase ProductoElectronica hereda de ProductoFisico y guarda su costo en dolares. Su metodo calcular_precio_final() multiplica ese valor por el dolar del dia. Ese valor lo entrega* ***servicios/indicador_dolar.py****, que consulta dos proveedores en cadena: primero mindicador.cl y, si no responde, findic.cl. En el pedido se usa ese precio congelado en la linea de detalle.*

Esos son los cinco requisitos funcionales. Miguel va a seguir ahora una operacion completa, desde el menu hasta la consulta SQL.

Bloque 3 (3:30 - 5:30) - Miguel: recorrido completo de una operacion

Vamos a seguir una sola operacion, de principio a fin: registrar el pago de un pedido. La idea es mostrar que cada capa tiene una responsabilidad distinta.

Paso 1, el menu. *En **main.py**, _registrar_pago() le pide al usuario que elija un pedido de la lista y llama al objeto del modelo. El menu no calcula nada: solo pide datos y arma el objeto.*

Paso 2, el modelo. *En **model/pedido.py**, registrar_pago() primero llama a confirmar_pedido(). Eso es importante: si no hay stock, la excepcion se lanza antes de tocar el inventario, y por eso el stock no cambia cuando la operacion se rechaza. Solo si la confirmacion pasa, se descuenta el stock de cada linea.*

Paso 3, los productos. *Cada producto descuenta su propio stock en descontar_stock(), en **model/producto.py**. Ese metodo vuelve a verificar el stock antes de rebajarlo: la regla esta en el modelo, no en la interfaz.*

Pasos 4 y 5, la persistencia. *El pedido queda con estado PAGADO y la capa dao lo guarda: actualizar_estado() en **dao/pedido_dao.py** escribe el estado y actualizar_stock() en **dao/producto_dao.py** escribe el nuevo stock de cada producto.*

Paso 6, la consulta SQL. *Esta es la sentencia que realmente guarda el cambio:*

```
UPDATE productos SET stock = :stock WHERE id_producto = :id
```

Los dos puntos indican un parametro enlazado. El valor que escribio el usuario viaja como dato, nunca como parte del texto de la sentencia. Eso es lo que evita la inyeccion SQL. Con esto, Alexandy muestra los requisitos no funcionales y la seguridad.

Bloque 4 (5:30 - 7:30) - Alexandy: no funcionales y seguridad

Ademas de los cinco requisitos funcionales, mostramos dos no funcionales.

Validacion de entradas. *Esta en **servicios/validador.py**. Las funciones pedir_entero(), pedir_decimal() y pedir_fecha_futura() piden el dato en un ciclo: si el valor no cumple el tipo, el formato o el rango, muestran el mensaje y vuelven a preguntar. El programa no se cae. Los setters del modelo validan lo mismo por su lado: stock no negativo, precio no negativo y fecha futura para un servicio. En el guion de pruebas, el caso P19 escribe letras donde va un numero y el caso P08 escribe un correo incompleto.*

Continuidad si la API falla. *Esta en **servicios/indicador_dolar.py**, con tres niveles. Primero, cada intento usa requests.get con timeout=5: el sistema no puede quedarse colgado esperando a un servidor lento. Segundo, si el proveedor principal no responde, el mismo codigo intenta con el proveedor de respaldo, que es findic.cl, y avisa cual se uso. Tercero, si fallan los dos, valor_para_calculo() usa el ultimo valor guardado en datos/dolar_respaldo.json y lo informa al operador. Eso es el caso P17: se corta el internet y el sistema informa y sigue funcionando.*

Consultas parametrizadas. *Estan en **dao/conexion.py** y en todos los DAO. Todas las sentencias usan parametros enlazados y la conexion activa PRAGMA foreign_keys = ON para que se respeten las claves foraneas. Si alguien escribiera comillas o un punto y coma en un campo, ese texto se guarda como dato y no se interpreta como SQL.*

Autenticacion y permisos. *En **model/trabajador.py** las contrasenas se guardan con PBKDF2-HMAC-SHA256 con sal aleatoria, nunca en texto plano. Y los permisos se resuelven por rol: el EncargadoBodega puede despachar pero no modificar el catalogo, y si lo intenta se lanza SinPermisoError. Miguel cierra con los cambios que hicimos despues de la retroalimentacion.*

Bloque 5 (7:30 - 9:30) - Miguel: cambios de la retroalimentacion y uso de IA

En la evaluacion anterior recibimos observaciones y las corregimos en el repositorio. Los cambios estan en el README, en la seccion de cambios incorporados.

El cambio principal es que los codigos ya no se escriben. *Antes habia que inventar el identificador del producto y del cliente, por ejemplo PROD-E01, y eso era lento y permitia repetir codigos. Ahora el sistema los genera: C01 para clientes y F01, D01, S01 y E01 para los cuatro tipos de producto. El codigo esta en* ***servicios/codigos.py****, en la clase GeneradorCodigos: consulta los codigos existentes, calcula el mayor y propone el siguiente, verificando que este libre antes de asignarlo.*

El segundo cambio es la interfaz. *Ahora los listados son tablas con encabezado y columnas alineadas, y se elige un registro por su numero en la lista, no escribiendo el codigo. Los formularios muestran el paso en el que van, por ejemplo [2 de 7], y al terminar aparece un panel con el resumen y el codigo asignado. Los mensajes se distinguen: [OK] para un exito, [!] para un aviso y [i] para informacion. Todo eso esta en* ***servicios/consola.py****.*

Sobre el uso de inteligencia artificial. *La usamos como apoyo y documentamos que hicimos con cada sugerencia. Adoptamos usar sqlite3.Row para leer las columnas por nombre en vez de por indice, porque asi un cambio en el orden del SELECT no rompe el codigo. Modificamos la sugerencia de consumir la API: la version original no tenia tiempo maximo de espera y la nuestra usa timeout y captura las fallas. Y descartamos dos: resolver el tipo de producto con if en vez de polimorfismo, porque la rubrica pide polimorfismo, y guardar la contrasena con MD5, porque no es seguro para contrasenas; usamos PBKDF2 con sal.*

Bloque 6 (9:30 - 10:00) - cierre

Para cerrar: los cinco requisitos funcionales y los dos no funcionales estan ubicados en el codigo, la operacion completa se puede seguir desde el menu hasta la consulta SQL, los cambios de la retroalimentacion estan en el repositorio y los datos estan protegidos con consultas parametrizadas, validacion de entradas y manejo de la falla de la API. El repositorio es publico y esta en pantalla. Gracias, quedamos atentos a sus preguntas.

4. Frases de transicion

Tener estas frases a mano evita silencios largos entre bloques.

Cuando	Frase
De arquitectura a requisitos	Con la arquitectura clara, vamos a ubicar los cinco requisitos de la ficha en el codigo.
De requisitos a recorrido	Ya sabemos donde esta cada requisito; ahora seguimos una operacion completa hasta la base de datos.
De recorrido a seguridad	Vimos como se guarda; ahora vemos como el codigo protege esos datos.
De seguridad a cambios	Ahora les mostramos que corregimos despues de la retroalimentacion de la evaluacion anterior.
De cambios al cierre	Con eso cerramos: requisitos ubicados, operacion recorrida, cambios incorporados y datos protegidos.
Si no saben algo	Eso no lo trabajamos de esa forma; en nuestro codigo se resuelve en la clase o funcion que les muestro ahora.

5. Banco de preguntas del docente

La defensa es individual: el docente elige una parte del codigo y pregunta a cada integrante por separado. Las respuestas deben ser cortas y siempre acompanadas del archivo en pantalla.

Pregunta	Respuesta preparada
Que pasaria si la consulta se armara concatenando texto en vez de usar parametros?	Se abre la puerta a la inyeccion SQL: si el usuario escribe comillas o un punto y coma, ese texto pasaria a formar parte de la sentencia y podria alterar o borrar datos. Con parametros enlazados el valor viaja como dato, nunca como instruccion. Se muestra en dao/conexion.py.
Que pasaria si se quita la validacion del RUT?	El setter aceptaria cualquier texto como identificador, se guardaria un cliente invalido y la regla de la ficha dejaria de cumplirse. La validacion esta en el setter, no en el menu, para que la proteccion no dependa de la interfaz.
Por que usaron excepciones propias y no un return False?	Porque una excepcion obliga a quien llama a manejarla y permite distinguir el motivo. Ademas la rubrica pide una excepcion propia por cada regla. Se capturan en main.py y el programa informa y continua.
Donde se descuenta el stock y por que en ese lugar?	En Pedido.registrar_pago(), despues de confirmar que hay stock, y en Producto.descontar_stock(). Si se descontara antes de confirmar, un pedido rechazado habria bajado el inventario igual.
Que pasa si la API del dolar no responde?	Hay tres niveles: cada intento tiene timeout de cinco segundos; si mindicador.cl no responde se intenta con findic.cl y se avisa cual se uso; si fallan los dos, valor_para_calculo() usa el ultimo valor guardado en datos/dolar_respaldo.json. El sistema nunca se cae: es el caso P17.
Por que usan dos proveedores de indicadores?	Porque el sistema no debe depender de un solo sitio externo. Los dos entregan el mismo formato de respuesta, asi que se recorren en orden con un for y el mismo parser sirve para ambos. Agregar un tercero es una linea mas en la tupla PROVEEDORES.
Por que el precio de la electronica no se guarda en pesos en el catalogo?	Porque depende del dolar del dia. Se guarda el costo en USD y se convierte al vender. El precio convertido queda congelado en la linea de detalle, para que el pedido no cambie si el dolar sube despues.
Que diferencia hay entre composicion y agregacion en su modelo?	Pedido y DetallePedido es composicion: la linea no existe sin el pedido y se elimina con el, declarado con ON DELETE CASCADE. DetallePedido y Producto es agregacion: si se elimina la linea o el pedido, el producto sigue en el catalogo.
Como evitan que dos productos tengan el mismo codigo?	GeneradorCodigos calcula el mayor numero usado y verifica con una consulta si el codigo candidato ya existe; si existe, prueba el siguiente. Por eso el codigo no se escribe a mano.
Que hace PRAGMA foreign_keys = ON?	Activa la revision de claves foraneas en SQLite. Sin eso se podria guardar un pedido con un cliente inexistente o una linea con un producto que no existe.
Que pasa si se elimina un pedido?	Se elimina la cabecera y, por la clausula ON DELETE CASCADE, tambien sus lineas de detalle. El producto sigue intacto porque es una agregacion.
Como se guardan las contrasenas?	Con PBKDF2-HMAC-SHA256, 120.000 iteraciones y una sal aleatoria de 16 bytes por usuario. La comparacion usa hmac.compare_digest. Nunca se guarda la contrasena en texto plano.
Que es un DAO y por que lo separaron del modelo?	Es la capa que guarda y lee datos. Se separa para que las clases del modelo no contengan SQL: asi se puede cambiar la base de datos sin tocar las reglas del negocio.

Pregunta	Respuesta preparada
Que parte del codigo propuso la IA y que hicieron con eso?	Esta en el README, seccion de uso de IA. Adoptamos el acceso por nombre de columna con sqlite3.Row; modificamos la consulta a la API agregando timeout y captura de errores; descartamos el if en vez de polimorfismo y el hash MD5.
Que pasaria si se elimina el timeout de la consulta a la API?	Si el servidor no responde, el programa quedaria esperando indefinidamente y el usuario no podria seguir. El timeout garantiza la continuidad del sistema, que es un criterio de la rubrica.
Por que reintentan en la validacion en vez de mostrar un mensaje y salir?	Porque la rubrica exige que un dato invalido se rechace con un mensaje sin detener el programa. El reintento permite corregir el dato y continuar la operacion.
Que cambiarian si tuvieran mas tiempo?	Agregar pruebas unitarias por clase, separar las credenciales en un archivo de configuracion y agregar un registro de auditoria de las operaciones criticas.

6. Fichas rapidas: si te toca explicar este archivo

Para cada archivo clave: que hace y que pasaria si se cambia algo. Es la parte que el docente pregunta de forma individual.

dao/conexion.py

Que hace: Define el esquema de las seis tablas, crea la base de datos al iniciar y ejecuta todas las consultas con parametros. El envoltorio de conexion cierra siempre el archivo para no dejar descriptores abiertos.

Si se cambia: Si se quita PRAGMA foreign_keys, se pueden guardar pedidos con clientes inexistentes. Si una consulta se arma con concatenacion en vez de parametros, se abre la inyeccion SQL.

dao/producto_dao.py

Que hace: Guarda los cuatro tipos de producto en una sola tabla con una columna tipo. Al leer, fila_a_producto() revisa el tipo y construye la instancia de la clase correcta.

Si se cambia: Si se agrega un tipo de producto nuevo, hay que agregar su caso en fila_a_producto y su valor permitido en el esquema; si no, la lectura falla.

dao/pedido_dao.py

Que hace: Guarda la cabecera del pedido y despues cada linea de detalle dentro de la misma conexion, con un solo commit. Ademas reconstruye el pedido con sus lineas al leerlo.

Si se cambia: Si el commit quedara dentro del ciclo de las lineas, un error a mitad de camino dejaria un pedido sin todas sus lineas.

servicios/validador.py

Que hace: Convierte y valida lo que escribe el usuario. validar_entero() rechaza texto y numeros fuera de rango; validar_rut() aplica Modulo 11; validar_fecha_futura() exige formato y fecha posterior a hoy. Las funciones pedir_* repiten la pregunta.

Si se cambia: Si se llamara a int() directamente sobre lo que escribe el usuario, el programa se cae con ValueError. Ese es el caso P19 del guion de pruebas.

servicios/indicador_dolar.py

Que hace: Recorre una cadena de proveedores (mindicador.cl y findic.cl) con timeout de cinco segundos cada uno, valida el codigo de respuesta y toma el valor mas reciente de la serie. Guarda ese valor como respaldo y entrega el ultimo conocido cuando fallan todos.

Si se cambia: Sin timeout el programa puede quedar colgado; sin capturar la excepcion, el sistema se cae cuando no hay internet. Si se quita un proveedor de la tupla se pierde esa red de seguridad.

servicios/codigos.py

Que hace: Genera los codigos de clientes y productos. Recorre los existentes con una expresion regular, toma el mayor y propone el siguiente, verificando que no exista antes de asignarlo.

Si se cambia: Si no se verificara la existencia, se intentaria insertar un codigo duplicado y la clave primaria rechazaria el registro.

model/cliente.py

Que hace: Modela al comprador. El atributo identificador se valida en el setter: RUT con Modulo 11 o correo con expresion regular. Si el dato es invalido, la excepcion se lanza al construir el objeto.

Si se cambia: Si la validacion se moviera al menu, cualquier otro programa que use la clase Cliente podria crear clientes invalidos.

model/pedido.py

Que hace: Es la transaccion: agrega lineas con el precio congelado, calcula el total con los fletes, confirma verificando el stock, registra el pago descontando el inventario y autoriza el despacho. Aqui viven las dos reglas de negocio.

Si se cambia: Si se descontara el stock antes de confirmar, un pedido rechazado por falta de stock habria bajado el inventario igual.

model/producto_fisico.py y sus subtipos

Que hace: ProductoFisico calcula el flete con peso, volumen y distancia. ProductoDigital y ProductoServicio sobrescriben procesar_entrega(). ProductoElectronica hereda de ProductoFisico y sobrescribe calcular_precio_final().

Si se cambia: Si se reemplazara el polimorfismo por un if sobre el tipo dentro de Pedido, cada tipo nuevo obligaria a modificar Pedido y la rubrica lo evalua como nivel En desarrollo.

7. Errores que hay que evitar

- Prometer que el codigo hace algo que no se puede mostrar en pantalla.
- Leer las diapositivas: el material de apoyo es el codigo, no la presentacion.
- Hablar de memoria sin abrir el archivo que se esta nombrando.
- Decir 'no se' sin ofrecer la parte que si se conoce: siempre se puede mostrar donde esta y para que sirve.
- Explicar las funciones del companero sin que le corresponda: se puede complementar, pero sin interrumpir.
- Usar terminos que no se puedan definir si el docente pregunta, por ejemplo polimorfismo o inyeccion SQL.

8. Cierre

La presentacion queda en diez minutos con este reparto y sobran cinco para las preguntas. Los cuatro puntos que hay que dejar dichos, si por tiempo algo queda fuera, son: donde esta cada requisito, como se sigue una operacion completa hasta la base de datos, que se corrigio despues de la retroalimentacion y como el codigo protege los datos.

La defensa individual depende de conocer los once archivos de la lista de pestanas. Con las fichas de la seccion 6 y el banco de preguntas de la seccion 5, cualquiera de los dos puede responder por cualquier parte del codigo.